Documentation technique du projet Gestion RH

Auteur : Ioty Randrianoely

Sommaire

- 1. Introduction
- 2. Résumé exécutif
- 3. Stack technique
- 4. Architecture globale
- 5. Approche méthodologique
- 6. Fonctionnalités principales
  - 6.1 Gestion des candidats
  - 6.2 Évaluation QCM et seuil d'accès à l'entretien
  - 6.3 Planification des entretiens
  - 6.4 Calendrier des entretiens
  - 6.5 Consultation des CV
- 7. Défis techniques rencontrés et solutions apportées
- 8. Résultats obtenus et points forts
- 9. Perspectives d'évolution

---

1. Introduction

Ce projet a pour objectif de digitaliser une partie du processus de recrutement au sein d'une structure RH. Il permet de gérer efficacement les candidats, de suivre leurs résultats à un QCM, de planifier des entretiens, d'annoter les évaluations et de consulter les documents associés tels que les CV.

L'application a été développée sous forme d'une application web moderne, avec une architecture backend robuste et une interface utilisateur claire, pensée pour faciliter le travail des recruteurs et des administrateurs.

---

2. Résumé exécutif

Le projet Gestion RH est une application web orientée recrutement, développée avec Spring Boot, Thymeleaf et PostgreSQL. Elle centralise les opérations clés du processus de sélection :

- gestion des candidats ;
- notation et suivi des résultats de QCM ;
- planification et suivi des entretiens ;
- consultation des supports de candidature ;
- visualisation des activités dans un calendrier.

L'objectif principal était de proposer un outil simple à utiliser, fiable et évolutif, capable de couvrir le parcours utilisateur du candidat jusqu'à la validation de l'entretien.

---

3. Stack technique

3.1 Backend

- Java 17
- Spring Boot 3.5.5
- Spring MVC
- Spring Data JPA
- Spring Web
- Maven

3.2 Frontend

- Thymeleaf
- HTML5
- CSS3
- JavaScript
- FullCalendar

3.3 Base de données

- PostgreSQL

3.4 Outils et dépendances

- OpenHTMLtoPDF pour la génération de documents PDF
- PostgreSQL JDBC Driver
- Spring Boot Test

3.5 Exemple de configuration de dépendances

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>

<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
</dependency>
```

---

4. Architecture globale

L'application suit une architecture en couches, typique d'une application Spring Boot.

4.1 Structure logique

- Controllers : réception des requêtes HTTP et orchestration des vues.
- Services : logique métier liée aux candidats, entretiens, scores et planning.
- Models : représentation des entités métier.
- Repositories / DAO : accès à la base de données via JPA.
- Templates Thymeleaf : rendu HTML côté interface utilisateur.

4.2 Principe de fonctionnement

Lorsqu'un utilisateur agit dans l'interface, la requête est transmise au contrôleur correspondant. Celui-ci appelle le service métier approprié, puis envoie les données à la vue Thymeleaf. Les données sont ensuite persister dans PostgreSQL.

---

5. Approche méthodologique

Le développement a été mené selon une approche itérative et orientée besoin métier. Les principales étapes ont été :

1. définition des besoins RH et flux de recrutement ;
2. modélisation des entités principales ;
3. implémentation des parcours essentiels (liste des candidats, QCM, planification) ;

4. intégration d’un calendrier pour la visualisation des entretiens ;
5. ajout de règles métier telles que le seuil de validation du QCM.

Cette approche a permis d’ajouter progressivement des fonctionnalités sans perdre la cohérence globale du système.

---

6. Fonctionnalités principales

6.1 Gestion des candidats

Cette fonctionnalité permet d’afficher la liste des candidats, d’appliquer des filtres, de voir leurs scores QCM ainsi que leur état de progression dans le processus de recrutement.

Le contrôleur centralise cette logique et prépare les données nécessaires à l’affichage dans la page de liste.

```
@GetMapping
public String list(Model model,
                   @RequestParam(required = false) String name,
                   @RequestParam(required = false) Double minScore,
                   @RequestParam(required = false) String profil,
                   @RequestParam(required = false) String propose) {

    List<Candidat> allCandidats = candidatService.getAll();
    if (allCandidats == null) allCandidats = List.of();

    Map<Integer, Double> scoresMap = new java.util.HashMap<>();
    for (Candidat c : allCandidats) {
        try {
            Integer annonceId = c.getAnnonce() != null ? c.getAnnonce().getId() : null;
            Double score = (annonceId != null) ? historiqueScoreService.getLatestScoreFor((long) c.getId(), annonceId) : null;
            scoresMap.put(c.getId(), score);
        } catch (Exception ex) {
            scoresMap.put(c.getId(), null);
        }
    }

    model.addAttribute("candidats", filtered);
    model.addAttribute("scoresMap", scoresMap);
    return "candidats/list";
}
```

Ce que cette fonctionnalité apporte

- consultation rapide de la liste des candidats ;
- filtres par nom, profil, score et statut ;
- meilleure visibilité sur l’avancement du recrutement.

6.2 Évaluation QCM et seuil d’accès à l’entretien

Le système permet d’évaluer les candidats au travers d’un QCM. Selon le score obtenu, l’accès à l’entretien peut être autorisé ou refusé. Cette règle métier est essentielle pour garantir un niveau minimum de préparation.

L’interface QCM utilise Thymeleaf et JavaScript pour afficher les questions, les options et les réponses correctes.

```
<ul class="options-list" th:id="'options-' + ${status.index}">
  <li th:each="option : ${question.options}" class="option-item">
    <label class="option-label" th:data-correct="${option.correct}">
      <input type="checkbox"
             class="option-checkbox"
             th:name="'answers[' + ${status.index} + '].selectedOptionIds'"
             th:value="${option.id}" />
      <span th:text="${option.text}">Option text</span>
      <span class="correct-mark">✓</span>
    </label>
  </li>
</ul>
```

Ce que cette fonctionnalité apporte

- parcours de validation interactif ;
- retour visuel sur les bonnes réponses ;
- contrôle métier de la progression vers l'entretien.

6.3 Planification des entretiens

La planification des entretiens est au cœur du projet. Deux modes sont disponibles :

- planification automatique ;
- planification manuelle.

Le contrôleur expose les routes permettant d'associer un candidat à un créneau et de sauvegarder ce planning en base.

```
@PostMapping("planifier_entretien_manual")
public String enregistrerPlaningManuel(@RequestParam int candidatId,
                                       @RequestParam String date,
                                       @RequestParam String time) {
    Candidat candidat = candidatService.getById(candidatId);
    if (candidat == null) {
        return "redirect:/candidats";
    }

    LocalDate d = LocalDate.parse(date);
    LocalTime t = LocalTime.parse(time);
    LocalDateTime debut = LocalDateTime.of(d, t);

    PlaningEntretien plan = new PlaningEntretien();
    plan.setCandidat(candidat);
    plan.setDateDebut(debut);
    plan.setDateFin(debut.plusHours(1));

    planningEntretienService.creerPlaningEntretien(plan);
    candidat.setEstPropose(true);
    candidatService.save(candidat);

    return "redirect:/candidats";
}
```

Ce que cette fonctionnalité apporte

- suivi des rendez-vous de recrutement ;
- meilleure organisation de l'équipe RH ;
- traçabilité de l'état du candidat.

6.4 Calendrier des entretiens

Le calendrier améliore la visibilité globale du planning. Il permet d'afficher les entretiens sous forme d'événements, avec un affichage journalier, hebdomadaire ou mensuel.

Le calendrier a été implémenté à partir de FullCalendar et rend les données récupérées via l'API de l'application.

```
this.calendar = new FullCalendar.Calendar(this.calendarEl, {
  initialView: 'listDay',
  locale: 'fr',
  headerToolbar: {
    left: 'prev,next today',
    center: 'title',
    right: 'listDay,listWeek,dayGridMonth'
  },
  events: this.fetchEvents.bind(this)
});
```

Ce que cette fonctionnalité apporte

- vision synthétique du planning ;
- navigation dans le temps ;
- meilleure coordination des équipes.

6.5 Consultation des CV

Les candidats disposent d'un support de candidature numérique. La plateforme permet d'afficher leurs CV directement depuis les ressources du projet.

```
@GetMapping("/cv/{filename:.+}")
public ResponseEntity<Resource> getCv(@PathVariable String filename) throws IOException {
    ClassPathResource file = new ClassPathResource("cv/" + filename);
    if (!file.exists()) {
        return ResponseEntity.notFound().build();
    }

    return ResponseEntity.ok()
            .header(HttpHeaders.CONTENT_DISPOSITION, "inline; filename=\"" + filename + "\"")
            .contentType(MediaType.parseMediaType("application/octet-stream"))
            .body(file);
}
```

Ce que cette fonctionnalité apporte

- accès rapide aux documents de candidature ;
- meilleure prise de décision pendant le tri des profils ;
- centralisation des informations.

---

7. Défis techniques rencontrés et solutions apportées

7.1 Gestion de la cohérence des données entre vues et base

Le projet a nécessité de synchroniser plusieurs sources d'informations : candidats, planning, scores QCM et état d'avancement. La solution a été de centraliser la préparation des données côté contrôleur et de transmettre des structures propres au template Thymeleaf.

7.2 Contrôle des règles métier

Une difficulté importante a été d'empêcher certaines actions si le candidat ne respectait pas les prérequis métier, comme un score QCM insuffisant. Cela a été traité par une logique conditionnelle claire dans l'interface et dans le backend.

7.3 Affichage dynamique du calendrier

L'intégration de FullCalendar a demandé une bonne compréhension du chargement asynchrone des événements et de la transformation des données métier en format exploitable par la bibliothèque.

7.4 Maintenance de l'interface et de l'expérience utilisateur

Certaines pages ont évolué avec de nombreuses contraintes de présentation. Le choix d'un style cohérent, d'un layout simple et d'un HTML bien structuré a permis d'obtenir une interface lisible et maintenable.

---

8. Résultats obtenus et points forts

Résultats obtenus

- application fonctionnelle et navigable ;
- flux recrutement partiellement automatisé ;
- interface claire pour le suivi des candidats ;
- intégration d'un planning dynamique ;
- mécanisme de validation QCM et d'éligibilité à l'entretien.

Points forts du projet

- architecture simple et extensible ;
- code organisé autour des concepts métier ;
- bonne séparation entre logique d'affichage et logique métier ;
- interface adaptée à un usage RH quotidien ;
- capacité d'évolution vers des modules plus avancés.

---

9. Perspectives d'évolution

Le projet peut être enrichi dans plusieurs directions :

- ajout d'un système d'authentification et de rôles utilisateurs ;
- intégration d'un tableau de bord RH plus complet ;
- gestion des notifications automatisées par email ;
- ajout de statistiques avancées sur les recrutements ;
- amélioration de la génération de rapports PDF ;
- extension vers une architecture plus modulaire avec API REST.

Cette évolution permettrait de transformer l'application en véritable plateforme de gestion de recrutement professionnelle, robuste et prête à un usage de production.

Conclusion

Ce projet illustre une solution web complète pour la gestion RH, à la fois fonctionnelle, pédagogique et adaptable. Il montre une bonne compréhension du développement Java / Spring Boot ainsi que de la logique métier liée à la gestion de recrutement.

Il constitue une base solide pour des améliorations futures et représente un outil pertinent dans un contexte professionnel réel.